§ 7. Персистентность: версии, стек и очередь

Обычная структура изменяет текущее состояние. Персистентная сохраняет доступ к старым состояниям после изменений. Можно прочитать прежнюю версию, продолжить вычисление от неё и одновременно оставить доступной другую ветвь истории. Для этого вовсе не обязательно копировать все данные после каждого действия.

В этом параграфе мы введём явный интерфейс версий, реализуем полностью персистентный стек и разберём очередь, для которой корректность сохраняется, а привычная амортизированная оценка — нет. Затем сравним копирование пути, толстые узлы и откат. Нужны связные списки, индексы узлов и доказательство очереди на двух стеках из § 1.

7.1. Что означает «старая версия доступна»

Пусть версия v_0 содержит последовательность $[5, 2]$. Из неё создадим v_1 добавлением 7 и независимо v_2 удалением последнего элемента. Получаются $[5, 2, 7]$ и $[5]$, но v_0 по-прежнему содержит $[5, 2]$. Команда изменения должна назвать исходную версию; команда чтения — нужную версию ответа.

Различают частичную персистентность: читать можно любую версию, изменять только последнюю; полную: изменять можно любую существующую версию; и конфлюэнтную: одна операция может объединять несколько версий. В последнем случае история зависимостей становится ориентированным ациклическим графом, а не деревом с единственным родителем каждой новой версии.

Копия номера версии не обязательно создаёт новые узлы. Две логические версии могут ссылаться на один корень и иметь одинаковое состояние. И наоборот, два разных корня могут представлять одинаковые последовательности. Разделяйте идентификатор истории, корень представления и логическое содержимое.

В этой главе все старые корни остаются доступными. Операции возвращают новый корень, вызывающий код хранит его под нужным номером. Мы не добавляем операцию удаления исторической версии и не обещаем автоматического освобождения переставших быть нужными узлов.

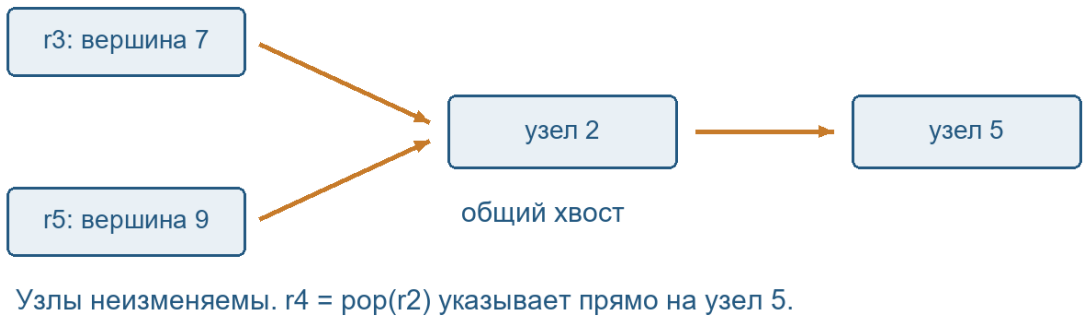


Рисунок 1: История операций ветвится; неизменяемые узлы разных версий могут иметь общий хвост.

Связь с видео. Запись 30.11.2022 посвящена сохранению версий; около 14:56 на кадре показано ветвление, а фрагмент 44:00–46:00 разбирает стек и расход памяти. Ниже формализуем интерфейс на C++17. Числа 5, 2 и 7 и имена версий выбраны для учебника.

(c) Aleksey Tolstikov

7.2. Неизменяемый список как представление стека

Стек хранится односвязным списком от вершины к основанию. Узел содержит значение и ссылку на следующий узел. Корень — ссылка на вершину, нулевой корень обозначает пустой стек. После создания поля узла не изменяются.

Для `push(r,x)` создаём один узел `(x,r)` и возвращаем его номер. Старый корень остаётся прежним: он не указывает на новый узел и не видит добавление. Для `pop(r)` возвращаем ссылку `next` узла r. Узлы не удаляются, поэтому старая версия продолжает читать прежнюю вершину.

Проследим историю. $r_0 = 0$; $r_1 = push(r_0, 5)$; $r_2 = push(r_1, 2)$; $r_3 = push(r_2, 7)$; $r_4 = pop(r_2)$; $r_5 = push(r_2, 9)$. Корни r_3 и r_5 ведут к разным вершинам 7 и 9, но дальше используют одни и те же узлы 2 и 5. Корень r_4 совпадает с r_1.

Версия	Последовательность от основания к вершине	Новых узлов
r_1	5	1
r_2	5, 2	1
r_3	5, 2, 7	1
r_4	5	0
r_5	5, 2, 9	1

Во всей истории создано четыре узла, хотя суммарная длина перечисленных непустых версий равна десяти. Экономия возникает из разделения общего хвоста. Узел не знает, сколько корней до него доходят: для чтения это не нужно.

Инвариант. Каждый ненулевой корень задаёт конечную цепочку неизменяемых узлов с нулём на конце. Новый узел ссылается только на ранее существовавший корень, поэтому цикл не возникает. Изменение возвращает новую цепочку или её хвост и не меняет цепочки старых корней. Индукция по числу созданных узлов доказывает корректность всех ветвей одновременно.

7.3. Реализация стека и время жизни узлов

Пул хранит узлы по целочисленным номерам. Это освобождает внешний интерфейс от сырых указателей. Нулевой элемент пула — служебный; действительные корни должны принадлежать именно этому экземпляру структуры. В примере используется `std::deque`, добавление в конец которого не копирует весь прежний массив узлов.

```
class PersistentStack {
public:
    using Root=std::size_t;
private:
    struct Node { long long value; Root next; std::size_t length; };
    std::deque<Node> pool{{0,0,0}}; // index 0 is the empty stack
public:
    bool empty(Root r) const { assert(r<pool.size()); return r==0; }
    std::size_t size(Root r) const {
        assert(r<pool.size()); return pool[r].length;
    }
```

В узле хранится длина хвоста вместе с вершиной. Поэтому `size` читает готовое число, а при добавлении достаточно увеличить длину исходной версии на единицу. Следующий листинг продолжает тот же класс; старые поля узлов не переписываются.

```
    Root push(Root r,long long value) {
        assert(r<pool.size());
        pool.push_back({value,r,pool[r].length+1});
        return pool.size()-1;
    }
    long long top(Root r) const {
        if (empty(r)) throw std::out_of_range("empty persistent stack");
        return pool[r].value;
    }
    Root pop(Root r) const {
        if (empty(r)) throw std::out_of_range("empty persistent stack");
        return pool[r].next;
    }
    std::size_t allocated() const { return pool.size()-1; }
};
```

Заголовок `code/07-persistence.hpp` содержит все необходимые включения. Исключение при чтении или удалении из пустого стека является частью контракта. `assert` контролирует принадлежность номера диапазону пула в отладочной сборке; он не отличит случайный допустимый номер из другого пула, поэтому такие номера запрещены предусловием.

`push` создаёт ровно один узел, `pop` не создаёт ни одного, `top`, `size` и `empty` читают постоянное число полей. Каждая операция имеет постоянное число действий на уровне узлов. Память после p добавлений — $O(p)$ узлов, плюс $O(v)$ для внешнего массива корней, если сохранено v именованных версий. Память нельзя оценивать только длиной текущего стека.

Пул живёт столько же, сколько объект `PersistentStack`. После уничтожения объекта все его номера теряют смысл. В нашей модели узлы не освобождаются по отдельности. Это простое и предсказуемое решение для конечной истории; система с долгим временем работы может потребовать сборки мусора или подсчёта ссылок.

Не следует хранить ссылку на элемент `std::vector`, затем добавлять в тот же вектор новые узлы и продолжать обращаться по старой ссылке: перевыделение делает её недействительной. Индексы сохраняют логическую идентичность узлов, но чтение по индексу всё равно должно происходить после потенциального перевыделения. Неизменяемость логических узлов не отменяет правил времени жизни объектов C++.

7.4. Список: цена изменения в середине

Стек особенно удобен, потому что его операции затрагивают голову списка. Если нужно изменить значение на расстоянии k от головы, старый узел переписывать нельзя: он доступен из прежних корней. Создадим новый узел с изменённым значением, а затем копии всех k предшествующих узлов с новыми ссылками. Неизменённый суффикс после позиции изменения разделяется со старой версией.

Таким образом, изменение по индексу требует $O(k+1)$ времени и новых узлов. Вставка и удаление в известной позиции также копируют путь от корня до места изменения. Наличие указателя на узел не позволяет бесплатно изменить входящую ссылку его предшественника во всех версиях.

Объединение двух списков через конкатенацию также зависит от представления. Чтобы приписать список B к хвосту A, нужно скопировать узлы A, изменив связь последнего узла, а затем разделить B. Цена $O(|A|)$, даже если оба входных корня уже существуют. Это пример операции с двумя версиями, но он не даёт универсальной константной конфлюэнтности.

Полное копирование списка перед каждым изменением было бы корректно, но стоило бы $O(n)$ независимо от позиции. Копирование пути сохраняет только изменённую часть достижимости. Для

сбалансированного дерева этот путь короток; для длинного списка — нет. Персистентность не исправляет плохую высоту исходного представления.

7.5. Очередь на двух персистентных стеках

Представим очередь парой стеков: `front` хранит начало очереди на вершине, `back` принимает новые элементы. Логическая последовательность — элементы `front` от вершины к основанию, затем элементы `back` от основания к вершине. Добавление создаёт новую вершину `back` и новую пару корней.

Если `front` пуст, перед чтением или удалением надо перевернуть весь `back` в новый стек. Старый входной стек сохраняется: операция создаёт новые узлы выходного стека, не переписывая старых. Версия очереди передаётся по значению, поэтому изменения локальной пары не меняют ранее сохранённые версии.

```
class PersistentQueue {
public:
    struct Version { PersistentStack::Root front=0, back=0; };
private:
    PersistentStack stack;
    Version normalize(Version v) {
        if (!stack.empty(v.front)) return v;
        while (!stack.empty(v.back)) {
            v.front=stack.push(v.front,stack.top(v.back));
            v.back=stack.pop(v.back);
        }
        return v;
    }
```

Публичные методы продолжают тот же класс. `normalize` — внутренний способ получить эквивалентное представление с непустым передним стеком, если очередь не пуста.

```
public:
    bool empty(Version v) const {
        return stack.empty(v.front) && stack.empty(v.back);
    }
    Version push(Version v,long long x) {
        v.back=stack.push(v.back,x); return v;
    }
    long long front(Version v) {
        v=normalize(v); return stack.top(v.front);
    }
    Version pop(Version v) {
        v=normalize(v); v.front=stack.pop(v.front); return v;
    }
    std::size_t allocated() const { return stack.allocated(); }
};
```

Код корректен как персистентная очередь: каждая возвращённая версия имеет нужную FIFO-последовательность, старые версии сохраняются. Но это **не** очередь с постоянной амортизированной стоимостью по произвольной истории версий. Даже чтение может создавать узлы, поскольку нормализованная пара из `front(v)` не сохраняется наружу.

На линейной цепочке добавлений и удалений, где результат каждого удаления используется дальше, переносы можно оплачивать как в § 1. Повторные чтения из одной ненормализованной версии в данном коде, однако, каждый раз выполняют перенос заново. Для них и для ветвящейся истории нужно отдельное рассуждение, которого обычная оценка двух стеков не предоставляет.

7.6. Почему потенциал нельзя потратить в каждой ветви

Создадим версию v с n элементами в `back` и пустым `front`. Теперь m раз выполним `pop(v)`, каждый раз начиная с одного и того же корня, и сохраним результаты как разные дочерние версии. Каждый вызов переворачивает n элементов, поэтому суммарная работа и число новых узлов равны $\Theta(mn)$.

В обычной очереди потенциал пропорционален размеру входного стека. Один дорогой перенос уменьшает его, и повторить этот перенос из прежнего состояния уже нельзя. При персистентности прежнее состояние доступно снова: одинаковый локальный запас пытаются использовать для оплаты работы во многих ветвях.

В формуле потенциалов из § 1 внутренние состояния сокращались вдоль одной цепочки. В дереве версий одна вершина может иметь несколько исходящих рёбер. Если независимо просуммировать разности потенциалов по рёбрам, потенциал ветвления будет вычтен несколько раз. Телескопическое сокращение больше не имеет прежнего вида.

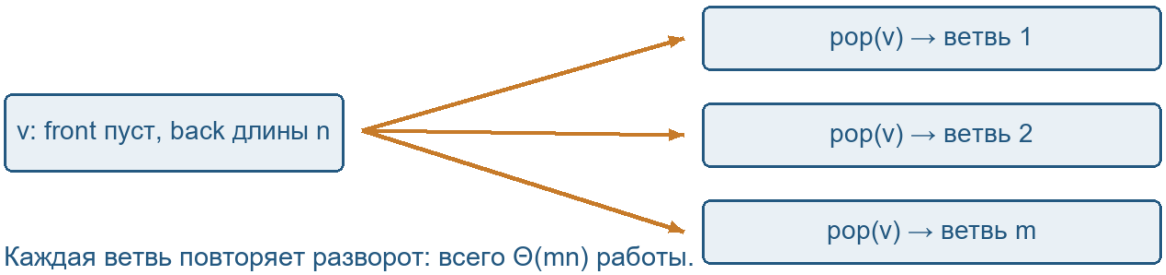


Рисунок 2: Несколько операций от одной версии повторяют один и тот же дорогой разворот.

Полезный тест производительности здесь детерминированный: построить описанную версию и многократно разветвиться именно от неё. Случайная история может почти никогда не выбирать такую вершину. В проверочном коде считаем новые узлы, а не только время; это непосредственно наблюдаемая стоимость, которую объясняет доказательство.

Мемоизация результата разворота может устранить повторение для одного и того же объекта, если кэш действительно разделяется между ветвями. Но тогда представление включает кэш, его заполнение, время жизни и гарантии однократного вычисления. В многопоточной системе добавляются вопросы синхронизации. Слова «лениво вычислим» без этой модели не являются доказательством.

Другой путь — очередь с постепенной перестройкой: дорогой разворот распределяется между будущими операциями, а состояние хранит расписание оставшейся работы. В конструкциях с несколькими стеками нужно доказать, что перестройка начинается достаточно рано, операция выполняет ограниченное число шагов, а начало очереди доступно до её завершения. Название «пять стеков» или «шесть стеков» само по себе не задаёт такой алгоритм. Здесь это направление углубления; приведённый код намеренно показывает простую корректную очередь и точную границу её эффективности.

7.7. Копирование пути и толстые узлы

Для дерева изменение одного листа затрагивает путь от корня. Создаём новый лист и новые копии его предков; неизменённые соседние поддеревья оставляем общими. Каждый скопированный узел указывает на правильные детей новой версии. Старый корень по-прежнему достигает только старых узлов или неизменённых общих поддеревьев.

Если высота равна h, число новых узлов и время изменения обычно $O(h)$ при постоянной работе в узле. В следующем параграфе дерево отрезков даст $h = O(\log n)$ без случайных предположений. Для обычного BST без балансировки высота может быть n; механическое копирование пути не даёт логарифмической гарантии.

Альтернативный подход — толстый узел: вместо перезаписи поля хранить историю его значений с номерами версий. При частичной персистентности и линейной истории можно искать последнее изменение не позже времени запроса. Однако поле с k изменениями требует поиска в его истории, а не бесплатного доступа к «старому значению».

Для полной персистентности линейного времени недостаточно. Пусть v_0 имеет поля $x = 0, y = 0$, из него создана v_1 с $x = 1$, а затем независимо v_2 с $y = 2$. Последнее хронологическое изменение x перед созданием v_2 произошло в v_1, но в v_2 значение x всё ещё равно 0: v_1 не является её предком. Нужно учитывать отношение предок–потомок, а не только сравнение номеров.

Общие преобразования структур в персистентные требуют дополнительных ограничений на число полей, входящие ссылки, способы копирования и поиск подходящих модификаций. Не следует переносить оценки простого стека на произвольный граф указателей. В этой главе доказана конкретная неизменяемая реализация, а не универсальный преобразователь.

7.8. Персистентность, снимки и откат

Полное копирование массива при каждом изменении создаёт снимки за $O(n)$ времени и памяти на версию. Это простой эталон корректности и иногда разумное решение для маленьких данных. Разделение узлов уменьшает стоимость локальных изменений, но требует дисциплины неизменяемости.

Откат хранит журнал изменений и умеет отменять последние действия. Если дерево истории известно заранее, можно обойти его в глубину: применить изменение при входе в вершину и откатить при выходе. В каждый момент существует одно текущее состояние. Это удобный offline-приём, но он не даёт произвольного одновременного доступа ко всем версиям через независимые корни.

Снимок диска, сохранение объекта в файл и персистентность в смысле структур данных — разные понятия. Наши версии существуют в оперативной памяти и исчезают вместе с процессом. Чтобы пережить сбой, нужны отдельные правила записи и восстановления; из неизменяемости узлов это автоматически не следует.

При выборе представления выпишите, какие версии будут читаться, разрешено ли ветвление, сколько операций ожидается и можно ли забывать историю. Если нужен только возврат на один шаг, журнал может быть проще. Если нужны независимые ветви и быстрые локальные изменения, корни и разделение подструктур дают естественный интерфейс.

7.9. Проверка ветвящейся истории и задания

Независимый эталон для стека — отдельный `std::vector` значений на каждую версию. Генератор выбирает любую старую версию, копирует эталон, выполняет изменение и сравнивает новую вершину, размер и содержимое. После создания потомка повторно проверяйте предка: иначе ошибка с изменением общего узла может долго оставаться незамеченной.

Для очереди используйте отдельную `std::deque` на версию. Проверяйте ветвление из пустой очереди, несколько удалений от одной версии, смешивание старых и новых ветвей и одинаковые значения. Для эффективности отдельно создайте контрпример из пункта 7.6; функциональный тест FIFO не обнаружит лишние копирования.

Практика. Добавьте стеку минимум, сохраняя его в каждом новом узле, и проверьте ветвление из версии с повторяющимися минимумами. Затем реализуйте копирование пути для точечного изменения в дереве из § 2. До написания кода укажите, какие узлы должны быть новыми, а какие разрешено разделить: это основа следующего параграфа о персистентном дереве отрезков.

7.10. Практика к главе

Прямой задачи на персистентный стек в просмотренных архивах iRunner не найдено. Авторское упражнение: добавьте минимум к стеку, создавайте новые версии из произвольных старых и сравнивайте ответы с полными копиями векторов. Проверьте пустую версию, одинаковые минимумы и неизменность предков.

7.11. Вопросы для самопроверки

1. Чем полная персистентность отличается от частичной?
2. Почему копия корня не равна полному копированию структуры?
3. Докажите, что новый узел стека не создаёт цикл.
4. Сколько новых узлов создают `push` и `pop`?
5. Почему память зависит от всей истории, а не от длины последней версии?
6. Сколько узлов нужно скопировать для изменения середины односвязного списка?
7. Почему очередь из листинга корректна, но не имеет заявленной для обычной очереди амортизации?
8. В каком месте ломается телескопическое суммирование по дереву версий?
9. Почему хронологический номер не заменяет отношение предка при чтении толстого узла?
10. Чем обход дерева истории с откатом отличается от доступных одновременно персистентных корней?

(c) Aleksey Tolstikov